SCAN-Planner: Spatial Collision-Aware Local Planning for Route-Guided Long-Range Quadruped Navigation

Han Zheng[1], Zhe Chen[1], Yiwen Fu[1], Ming Yang[1], and Tong Qin[1*]

***Abstract*—Quadruped robots are increasingly expected to navigate through narrow passages, cluttered indoor scenes, and large-scale 3D unstructured environments. Existing local planners commonly approximate the robot using isotropic geometric inflation or rely on planar and elevation-map representations, leading to conservative motion in tight spaces and limited reasoning about overhanging structures. This letter presents SCAN-Planner, a spatial collision-aware local planning framework for long-range quadruped navigation. A yaw-aware twin-cylinder footprint is used to model the elongated robot body, enabling whole-body collision evaluation through sparse queries in an inflated 3D occupancy map. We further introduce a projected A* search that generates collision-free guidance on an interpolated ground-following surface, with z-gradient suppression to avoid obstacles horizontally while maintaining vertical stability. For large-scale deployment, a robot-centric sliding map with boundary fallback provides high-resolution local collision checking and recovery from local dead ends. Simulation and real-world experiments demonstrate that SCAN-Planner generates safe, smooth, and efficient trajectories in dense clutter, 3D unstructured scenes, stair traversal, and long-range navigation tasks. The code will be released at https://github.com/wuyi2121/SCAN-Planner.**

***Index Terms*—Trajectory optimization, whole-body collision checking, B-spline planning, quadruped robots.**

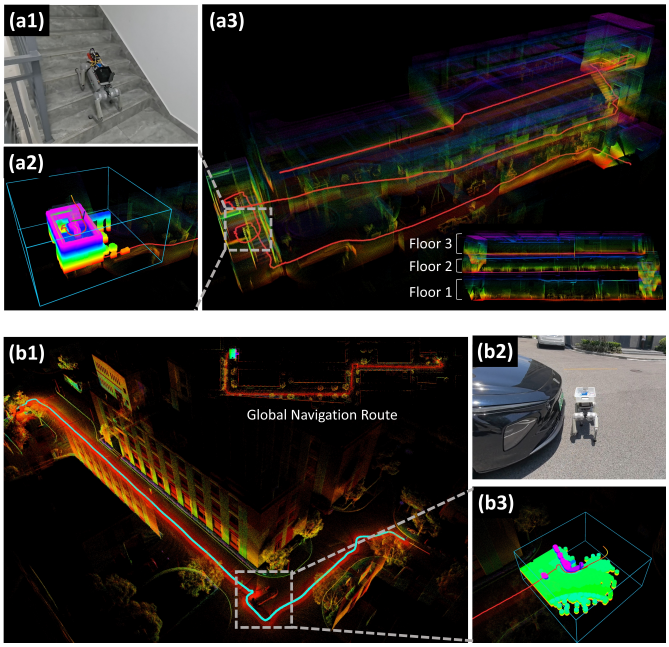

Fig. 1: Real-world demonstrations of SCAN-Planner in challenging quadruped navigation tasks. (a1)–(a3) Multi-floor indoor navigation: the robot traverses staircases across different floors while performing local 3D collision checking, and the reconstructed point-cloud map shows the complete cross-floor trajectory. (b1)–(b3) Long-range outdoor navigation: the robot follows a coarse global navigation route in a large-scale campus environment, while SCAN-Planner maintains a robot-centric local occupancy map for real-time obstacle avoidance and route-guided trajectory generation.

I. Introduction

Quadruped robots are increasingly deployed for autonomous inspection, delivery, and service tasks in environments where wheeled robots are limited by discontinuous terrain and aerial robots are constrained by payload, endurance, or safety requirements. Recent systems have demonstrated strong capabilities in field navigation [1], multi-goal planning [2], and perceptive locomotion over rough terrain [3], but they typically address terrain traversability, locomotion stability, or local obstacle avoidance as separate problems. In practical long-range navigation, however, a quadruped robot must move through narrow passages, cluttered indoor scenes, staircases, and large-scale unstructured environments within a single planning framework. This setting makes local planning particularly challenging: the planner must account for the yaw-dependent body footprint in tight spaces, reason about full 3D obstacles while preserving ground-following motion, and remain scalable when only a local map is available under coarse global route guidance.

The first difficulty lies in whole-body collision checking in narrow spaces. Most real-time local planners achieve efficiency by modeling the robot as a point, sphere, or circle and inflating obstacles accordingly [4]–[6]. Such a simplification is effective for compact robots, but it is inaccurate for quadruped bodies whose collision region changes with yaw. A width-based inflation may be unsafe during turning, whereas a length-based inflation easily becomes over-conservative and blocks feasible narrow passages. Whole-body planners have been developed to improve collision fidelity. For example, RC-ESDF [7] prebuilds a robot-centric signed distance field and jointly optimizes position and rotation, while 3D RoA-Planner [8] searches collision-free yaw ranges for confined-space quadruped planning. Nevertheless, explicit yaw-state reasoning or rotation optimization increases planning complexity. For onboard replanning, it is desirable to retain a compact position-trajectory optimization while evaluating collision safety with a yaw-aware body footprint.

The second difficulty comes from 3D unstructured scenes. A 2D planner cannot distinguish a wall from a traversable step, and a 2.5D elevation map cannot faithfully represent overhanging tables, shelves, or multi-level structures around the robot body. Recent ground-robot planners exploit richer 3D information, such as traversable planes [9] or multilevel terrain representations [10]. However, directly treating a ground robot

[1]Shanghai Jiao Tong University, Shanghai, China. {hanzheng, qintong}@sjtu.edu.cn.
[*] is the Corresponding author. This work was supported by the Natural Science Foundation of Shanghai (Grant No. 24ZR1435600).

TABLE I: Capability Comparison for Recent Local Planners

Methods	Terrain Traversal	Overhang Obstacles	Ground Following	Smooth Trajectory	Long-Range Navigation	Yaw-Aware Body
EGO-Planner-2D [4]	×	×	✓	✓	×	×
EGO-Planner-3D [4]	✓	✓	×	✓	×	×
ART-Planner [1]	✓	×	✓	×	×	✓
CMU-Planner [11]	✓	×	✓	×	×	×
SCAN-Planner (Ours)	✓	✓	✓	✓	✓	✓

as a free-flying 3D body can introduce unnecessary vertical detours during local optimization. For stable locomotion, obstacle avoidance should mainly deform the trajectory horizontally while preserving the vertical trend on slopes or stairs.

The third difficulty is long-range navigation with a local planner. A fixed-horizon planner is efficient, but it is prone to local minima and dead ends when the target is far away and the map is only partially observed. Large-scale autonomy systems therefore often rely on hierarchical guidance or coverage paths [11, 12], while robot-centric occupancy maps reduce memory cost by maintaining a local grid around the robot [13]. However, route guidance, sliding-map boundaries, high-resolution collision checking, and dead-end recovery must be handled consistently. Otherwise, a local planner may fail when the subgoal lies behind a newly observed obstacle or outside the current map boundary.

To bridge these gaps, we propose SCAN-Planner, a spatial collision-aware local planner for route-guided long-range quadruped navigation. SCAN-Planner keeps the efficiency of position-only B-spline optimization, while making initialization, collision segmentation, rebound search, safety checking, and execution consistent with a yaw-aware quadruped footprint. It further combines a projected A* search with z-gradient suppression, a robot-centric sliding occupancy map, and global coarse-route guidance to support narrow-space, 3D, and long-range navigation. The main contributions of this paper are summarized as follows:

- We propose a yaw-aware footprint representation for efficient whole-body collision checking of quadruped robots in narrow and 3D unstructured spaces.
- We introduce a projected A* planning scheme with z-gradient suppression, enabling horizontal obstacle avoidance while maintaining vertical stability in 3D planning.
- We develop a robot-centric sliding map that supports high-resolution local collision checking, scalable planning in large-scale environments, and robust dead-end recovery.
- We integrate local trajectory optimization with global coarse-route guidance, and validate the reliability and generalizability of the system through extensive real-world experiments.

II. Related Work

A. Local Trajectory Planning

Local trajectory planning in cluttered environments is commonly addressed through kinodynamic search, corridor-constrained optimization, and gradient-based trajectory deformation. Kinodynamic planners generate dynamically feasible trajectories by searching in the state space [5, 6], while corridor-based methods optimize polynomial or MINCO trajectories within safe convex regions [14, 15]. Gradient-based replanning methods further improve efficiency by optimizing smoothness, feasibility, and collision costs. RAPTOR [16] improves replanning quality with topological path guidance and perception-aware refinement, while EGO-Planner [4] avoids explicit ESDF construction by lazily generating rebound constraints from collision-free guiding paths. These methods provide efficient local trajectory generation in cluttered environments and have been widely adopted for real-time robotic navigation.

Despite their efficiency, most local planners rely on point-mass or isotropically inflated robot models for collision checking. Such abstractions are insufficient for elongated quadruped bodies whose footprint varies with yaw, especially in narrow spaces with lateral or overhanging obstacles. Whole-body collision-aware methods, such as RC-ESDF [7], improve collision fidelity by reasoning over robot-centric distance fields and orientation states, but the additional state dimensions increase onboard replanning complexity. Efficient quadruped local planning therefore requires a collision model that is both body-aware and lightweight for real-time optimization in 3D cluttered environments.

B. Quadruped Robot Navigation

Quadruped navigation requires terrain perception, locomotion feasibility, and navigation-level planning. Existing legged navigation methods estimate traversable regions using reachability reasoning, template learning, or elevation mapping [17, 18]. Other works learn policy-specific traversability from volumetric occupancy [19], improve terrain-adaptive control with nonlinear MPC [3], or train learned policies for challenging terrain traversal [20]. These methods provide strong locomotion and traversability capabilities, but they mainly focus on local terrain feasibility rather than smooth trajectory planning in cluttered 3D spaces. As a result, their outputs are not directly formulated as continuous, collision-aware local trajectories for navigation.

Navigation-oriented systems further incorporate terrain and body constraints into higher-level planning. ART-Planner [1] combines traversability-informed sampling with MPC for robust field navigation, while SMUG Planner [2] performs safe multi-goal planning with robot-specific validity checking. Recent methods also exploit foot-end terrain features [21] or search collision-free yaw ranges for confined-space maneuvering [8]. However, deploying quadruped navigation in large-scale environments remains challenging under bounded-memory local mapping. The finite map window may truncate obstacles and route information at the map boundary, while long-range route following inevitably encounters dead-end regions where local planners can become trapped in local minima.

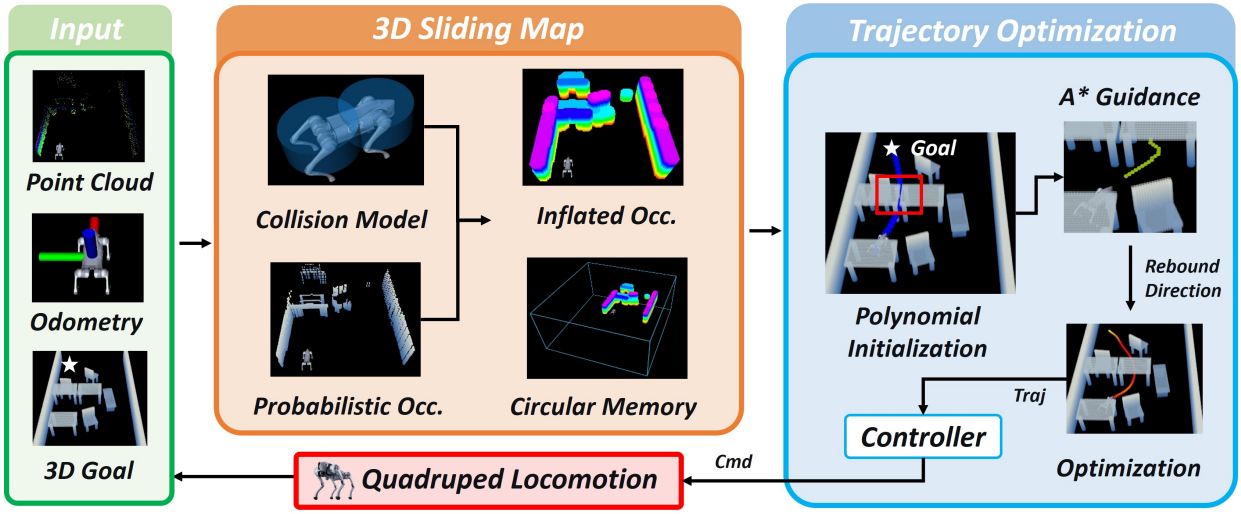

Fig. 2: System architecture of SCAN-Planner. The system takes point clouds, odometry, and a route-guided 3D local goal as inputs. A robot-centric 3D sliding map maintains probabilistic occupancy, yaw-aware inflated occupancy, and circular-buffer memory for high-resolution local collision checking. Based on the map representation, the trajectory optimization module generates a height-regularized polynomial initialization, performs projected A* guidance and rebound-direction construction when collisions are detected, and optimizes a smooth collision-free trajectory for execution by the quadruped locomotion controller.

III. System Overview

The proposed system is shown in Fig. 2. SCAN-Planner takes the point cloud, odometry, and a 3D goal from the global route as inputs. A robot-centric 3D sliding map maintains probabilistic occupancy, inflated occupancy, and circular-buffer memory for high-resolution local collision checking and large-scene operation. The trajectory planning module first generates a height-regularized polynomial initialization, detects unsafe segments using the yaw-aware footprint, searches for projected A* guidance when needed, and optimizes a smooth B-spline trajectory. The optimized trajectory is sent to the controller and executed by the quadruped locomotion module.

IV. Yaw-Aware Local Planning

A. Trajectory Representation

For a quadruped robot whose heading can be decoupled from its direction of motion, we optimize only a position trajectory $\mathbf{p}(t) \in \mathbb{R}^3$, and assign an induced yaw angle $\psi(t) \in \mathbb{S}^1$ for footprint collision checking and execution. Specifically, the position trajectory is parameterized by a uniform B-spline with degree p_b, knot span Δt, and N_c control points $\{\mathbf{Q}_i\}_{i=0}^{N_\mathrm{c}-1}$, which is a piecewise polynomial uniquely determined by its control points. In practice, we use a cubic B-spline with $p_\mathrm{b} = 3$, whose duration is $T = (N_\mathrm{c} - p_\mathrm{b})\Delta t$. The control points of the velocity, acceleration, and jerk curves are obtained by:

$$\mathbf{V}_i = \frac{\mathbf{Q}_{i+1} - \mathbf{Q}_i}{\Delta t}, \ \mathbf{A}_i = \frac{\mathbf{V}_{i+1} - \mathbf{V}_i}{\Delta t}, \ \mathbf{J}_i = \frac{\mathbf{A}_{i+1} - \mathbf{A}_i}{\Delta t}. \tag{1}$$

We optimize a subset of $N_\mathrm{c} - 2p_\mathrm{b}$ interior control points, $\{\mathbf{Q}_{p_\mathrm{b}}, \mathbf{Q}_{p_\mathrm{b}+1}, \ldots, \mathbf{Q}_{N_\mathrm{c}-p_\mathrm{b}-1}\}$. The first and last p_b control points are kept fixed because they determine the boundary state.

For a quadruped robot, aligning the body heading with the local direction of motion reduces the lateral footprint in narrow passages. Therefore, instead of optimizing yaw independently, we induce the yaw used for whole-body collision checking from the local tangent of the position trajectory:

$$\psi_i = \operatorname{atan2}\left(\mathbf{e}_\mathrm{y}^\top(\mathbf{Q}_{i+1} - \mathbf{Q}_{i-1}), \mathbf{e}_\mathrm{x}^\top(\mathbf{Q}_{i+1} - \mathbf{Q}_{i-1})\right). \tag{2}$$

where $\mathbf{e}_\mathrm{x} = [1,0,0]^\top$ and $\mathbf{e}_\mathrm{y} = [0,1,0]^\top$. Here, $\mathbf{Q}_{i+1} - \mathbf{Q}_{i-1}$ provides a finite-difference approximation of the local motion direction associated with the i-th checking point, as shown in Fig. 3. This coupling avoids introducing additional yaw control variables while keeping the footprint collision check consistent with tangent-following quadruped execution.

B. Whole-Body Collision Check

To improve efficiency, we approximate the elongated robot body by two vertical cylinders fixed in the body frame. After inflating the occupancy map by the cylinder radius and vertical clearances, yaw-aware whole-body collision checking reduces to finite-point queries of the transformed cylinder centers. As shown in Fig. 3, the two cylinder centers are placed along the body longitudinal axis:

$$\mathbf{s}_{\mathrm{b},1} = [d_\mathrm{off}, 0, 0]^\top, \qquad \mathbf{s}_{\mathrm{b},2} = [-d_\mathrm{off}, 0, 0]^\top, \tag{3}$$

where d_off is the longitudinal offset from the robot center. Each cylinder is parameterized by a horizontal clearance d_xy, an upward clearance d_up, and a downward clearance d_down; d_step denotes the maximum traversable step height used to set the lower clearance. The upward clearance d_up is selected to cover the maximum vertical extent of the robot body and mounted hardware with a small safety margin, enabling the planner to distinguish truly blocking overhead obstacles from structures with sufficient clearance. The downward clearance d_down is set to the nominal robot-center height minus d_step, thereby excluding traversable low obstacles from the inflated region while still rejecting obstacles that exceed the robot's terrain-crossing capability.

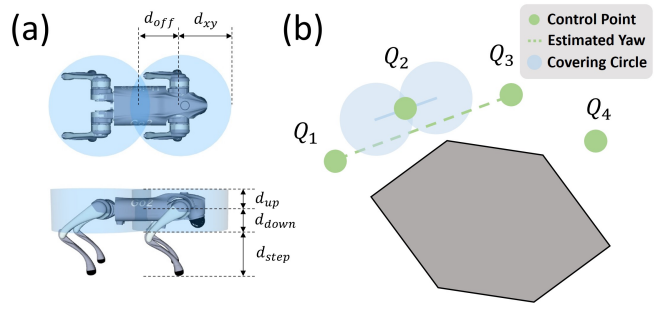

Fig. 3: Yaw-aware footprint for whole-body collision checking. (a) The quadruped body is covered by two vertical cylindrical primitives with longitudinal offset and clearance parameters. (b) Along the B-spline control points, collision checking is reduced to querying the transformed cylinder centers in the inflated occupancy map.

Given a checking control point $\mathbf{Q}_k$ and its induced yaw ψ_k, the cylinder centers in the world frame are

$$\mathbf{s}_{k,j} = \mathbf{R}_z(\psi_k)\mathbf{s}_{\mathrm{b},j} + \mathbf{Q}_k, \quad j \in \{1, 2\}. \tag{4}$$

We maintain a 3D occupancy map inflated according to the above cylindrical primitive. Thus, whole-body collision checking only requires querying the two transformed cylinder centers:

$$H_k(\mathbf{Q}_k, \psi_k) = \max_{j\in\{1,2\}} \chi(\mathbf{s}_{k,j}), \tag{5}$$

where $\chi(\cdot)$ denotes the inflated occupancy value. The robot is considered collision-free at the checking control point $\mathbf{Q}_k$ if $H_k(\mathbf{Q}_k, \psi_k) = 0$.

C. Objective Functions

The optimization problem is formulated as follows:

$$\min_{\mathbf{Q}} J = \lambda_c J_c + \lambda_s J_s + \lambda_f J_f, \tag{6}$$

where J_c is the rebound collision penalty, and J_s and J_f denote the smoothness and feasibility penalties, respectively. The coefficients λ_c, λ_s, and λ_f are the corresponding weights.

1) Height-Regularized Initialization: The initial path is generated from a polynomial trajectory. To avoid unnecessary vertical oscillation in 3D planning, we regularize the height of the sampled path before B-spline parameterization. Let $\mathcal{R} = \{\mathbf{r}_0, \mathbf{r}_1, \ldots, \mathbf{r}_{N_p}\}$ be the raw samples of the polynomial path. We first compute the accumulated horizontal arc length:

$$\ell_i = \sum_{m=1}^{i} \left\|\mathbf{r}_{m,\mathrm{xy}} - \mathbf{r}_{m-1,\mathrm{xy}}\right\|, \quad L = \ell_{N_p}. \tag{7}$$

Then the height assigned to the i-th sample is

$$z_i = z_s + \alpha_i(z_g - z_s), \quad \alpha_i = \begin{cases} \ell_i/L, & L > 0, \\ i/N_p, & L = 0, \end{cases} \tag{8}$$

where z_s and z_g are the start and local target heights. The height-regularized path preserves the xy-projection of the initial path, while its z-coordinate follows a linear profile along the accumulated horizontal arc length. These height-adjusted samples are used to parameterize the B-spline, providing a stable vertical profile before rebound optimization. As shown in Fig. 4, this encourages obstacle avoidance mainly through lateral deformation while suppressing unnecessary vertical oscillation.

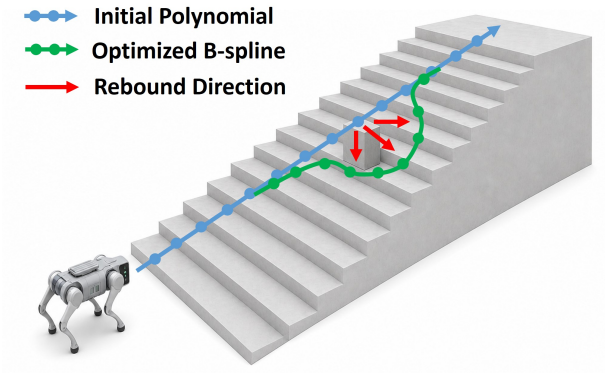

Fig. 4: Projected rebound deformation with z-gradient suppression on stair-like terrain. The height-regularized polynomial initialization (blue) provides the nominal vertical profile, while the optimized B-spline (green) avoids the obstacle through horizontal deformation guided by rebound gradients (red), reducing unnecessary vertical oscillation.

2) Collision Penalty: The collision penalty is constructed only for colliding segments detected by the yaw-aware footprint check in Eq. (5). For each colliding segment, a collision-free path Γ is generated by the projected A* search. Following the rebound formulation [4], each affected control point $\mathbf{Q}_i$ is assigned one or more $\{\mathbf{a}, \mathbf{v}\}$ pairs, where $\mathbf{a}_{ij}$ is an anchor point and $\mathbf{v}_{ij}$ is the corresponding unit rebound direction. The obstacle distance from $\mathbf{Q}_i$ to the j-th obstacle is defined as

$$d_{ij} = (\mathbf{Q}_i - \mathbf{a}_{ij})^\top \mathbf{v}_{ij}. \tag{9}$$

The collision penalty is then written as

$$J_c = \sum_{i,j} \rho(c_{ij}), \tag{10}$$

where $c_{ij} = s_f - d_{ij}$, s_f is the safety clearance, and $\rho(\cdot)$ is a twice continuously differentiable one-sided penalty. The corresponding gradient is

$$\frac{\partial J_c}{\partial \mathbf{Q}_i} = -\sum_j \rho'(c_{ij})\mathbf{v}_{ij}. \tag{11}$$

The binary occupancy indicator in Eq. (5) is not differentiated; it is used only to detect colliding segments and lazily construct the $\{\mathbf{a}, \mathbf{v}\}$ pairs needed for the smooth collision penalty.

3) Smoothness and Feasibility Penalty: The smoothness penalty minimizes the squared norm of jerk control points:

$$J_s = \sum_k \|\mathbf{J}_k\|^2. \tag{12}$$

To limit the velocity and acceleration along the trajectory, the feasibility penalty uses a one-sided squared penalty and is written as:

$$J_f = \sum_k \mathcal{F}(\|\mathbf{V}_k\|, v_m) + \sum_k \mathcal{F}(\|\mathbf{A}_k\|, a_m), \tag{13}$$

where v_m and a_m are the maximum velocity and acceleration, respectively. The function $\mathcal{F}(c, c_m) = \max(c - c_m, 0)^2$ is a one-sided squared penalty. In implementation, the vertical component of the optimization update is suppressed, encouraging obstacle avoidance mainly through horizontal deformation. Since yaw is induced from the position trajectory, no yaw control points are optimized. During execution, the physical heading is regulated to track the planned tangent direction.

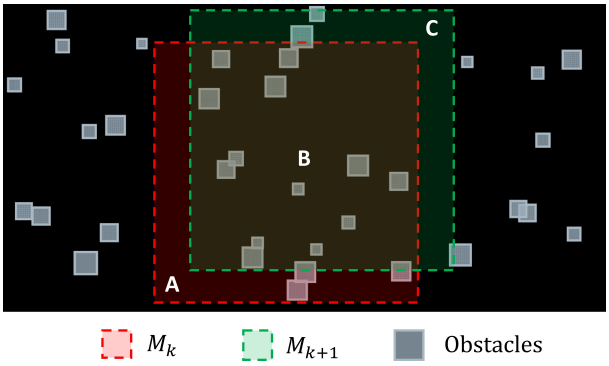

Fig. 5: Sliding map update. When the active map shifts from $\mathcal{M}_k$ (red) to $\mathcal{M}_{k+1}$ (green), the overlapped region B is maintained without memory relocation, the outgoing region A is released, and the incoming region C is reset for newly observed voxels.

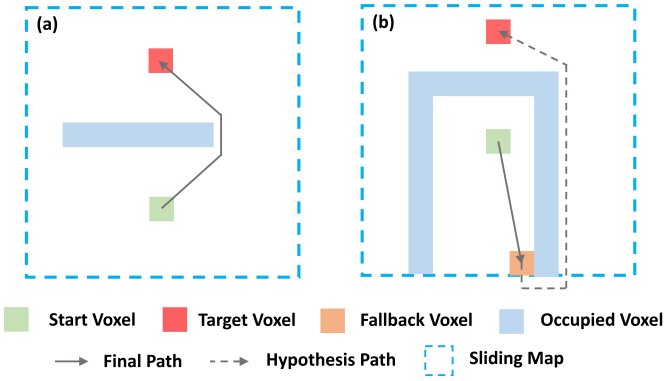

Fig. 6: Projected A* guidance and boundary fallback. (a) When the target is reachable inside the sliding map, the final path detours around occupied voxels. (b) In a dead-end case, a hypothesis path is allowed to enter the virtual free layer outside the map boundary, and the exit boundary voxel is selected as a fallback target for a bounded recovery motion.

V. 3D Sliding Map

A. Sliding Strategy

The local map is maintained as a uniform 3D occupancy grid. Following common occupancy-mapping practice [13, 22], raycasting updates the log-odds occupancy value of each voxel with bounded probability clamping. The upper clamp $p_{\max}$ prevents occupied voxels from becoming over-confident, while the lower clamp $p_{\min}$ allows free or unknown regions to be updated rapidly when new observations arrive. This update mechanism supports both persistent structures and transient obstacles in complex environments.

The map is implemented as a fixed-size sliding window centered around the robot. As illustrated in Fig. 5, when the sliding map shifts from $\mathcal{M}_k$ to $\mathcal{M}_{k+1}$, the overlapping region B is preserved, while the outgoing region A releases circular-buffer addresses that are reused by the incoming region C. Only the recycled addresses are reset and reinitialized, and the buffer contents associated with the overlapping region are kept unchanged. Following ROG-Map [13], the inflated occupancy layers are updated incrementally when voxel occupancy states change rather than rebuilding the whole inflated map. This zero-copy sliding mechanism keeps memory bounded and enables high-resolution local mapping during long-range navigation.

B. Projected A* Guidance

When a colliding segment is detected, a collision-free path Γ is searched between the entry point $\mathbf{x}^{\text{in}}$ and the exit point $\mathbf{x}^{\text{out}}$ to construct the $\{\mathbf{a}, \mathbf{v}\}$ pairs. Instead of searching the full 3D grid, we restrict A* expansion to an inclined search surface whose height is interpolated from the two endpoints. For a candidate horizontal grid point $\mathbf{x}_{\text{xy}}$, the interpolation ratio is

$$\beta(\mathbf{x}_{\text{xy}}) = \Pi_{[0,1]}\left(\frac{(\mathbf{x}_{\text{xy}} - \mathbf{x}_{\text{xy}}^{\text{in}})^{\top}(\mathbf{x}_{\text{xy}}^{\text{out}} - \mathbf{x}_{\text{xy}}^{\text{in}})}{\|\mathbf{x}_{\text{xy}}^{\text{out}} - \mathbf{x}_{\text{xy}}^{\text{in}}\|^2}\right), \tag{14}$$

where $\Pi_{[0,1]}(\eta) = \min\{1, \max\{0, \eta\}\}$ denotes projection onto the interval $[0, 1]$. The corresponding height is

$$z(\mathbf{x}_{\text{xy}}) = z^{\text{in}} + \beta(\mathbf{x}_{\text{xy}})(z^{\text{out}} - z^{\text{in}}). \tag{15}$$

Thus, each search node is represented by its horizontal grid coordinate and the interpolated height.

During neighbor expansion, we set the checking yaw to $\psi_\Delta = \text{atan2}(\Delta y, \Delta x)$, and accept a candidate node only when the yaw-aware collision indicator in Eq. (5) is zero. This projected search suppresses unnecessary vertical detours and keeps the rebound guidance consistent with ground-robot motion.

C. Dead-End Recovery

To improve robustness in dead-end cases, we further adopt a boundary fallback strategy, as illustrated in Fig. 6(b). During the rebound search, the sliding map is temporarily augmented with a one-voxel-wide virtual free layer outside its boundary. This layer is used only for hypothesis-path generation and is not regarded as executable free space. If the original target cannot be reached inside the sliding map, the projected A* search is allowed to enter the virtual layer, and the boundary voxel where the hypothesis path leaves the sliding map is selected as a fallback voxel. The local target is then replaced by this fallback voxel, and the final path is generated within the real sliding map using the yaw-aware twin-cylinder collision check. This mechanism converts a dead-end planning failure into a bounded recovery motion toward the map boundary.

VI. Experimental Results

A. Experimental Setup

We evaluate SCAN-Planner against four representative local planning baselines using the MARSIM simulator [23]: EGO-Planner-2D and EGO-Planner-3D [4], CMU-Planner [11], and ART-Planner [1]. EGO-Planner-2D denotes the planar implementation of EGO-Planner adapted for ground-robot navigation. The evaluation covers three representative scenarios: a random forest, a cluttered desk scene, and a stair scene. Each method is tested over 50 independent trials per scene, with maximum velocity 1.5 m/s, maximum acceleration 1.0 m/s^2, and a timeout of 120 s. All simulations are run on an Intel Core i9-14900K CPU. The B-spline optimization problems are solved using L-BFGS [24]. We report success rate (SR), collision rate (CR), and success weighted by path length (SPL).

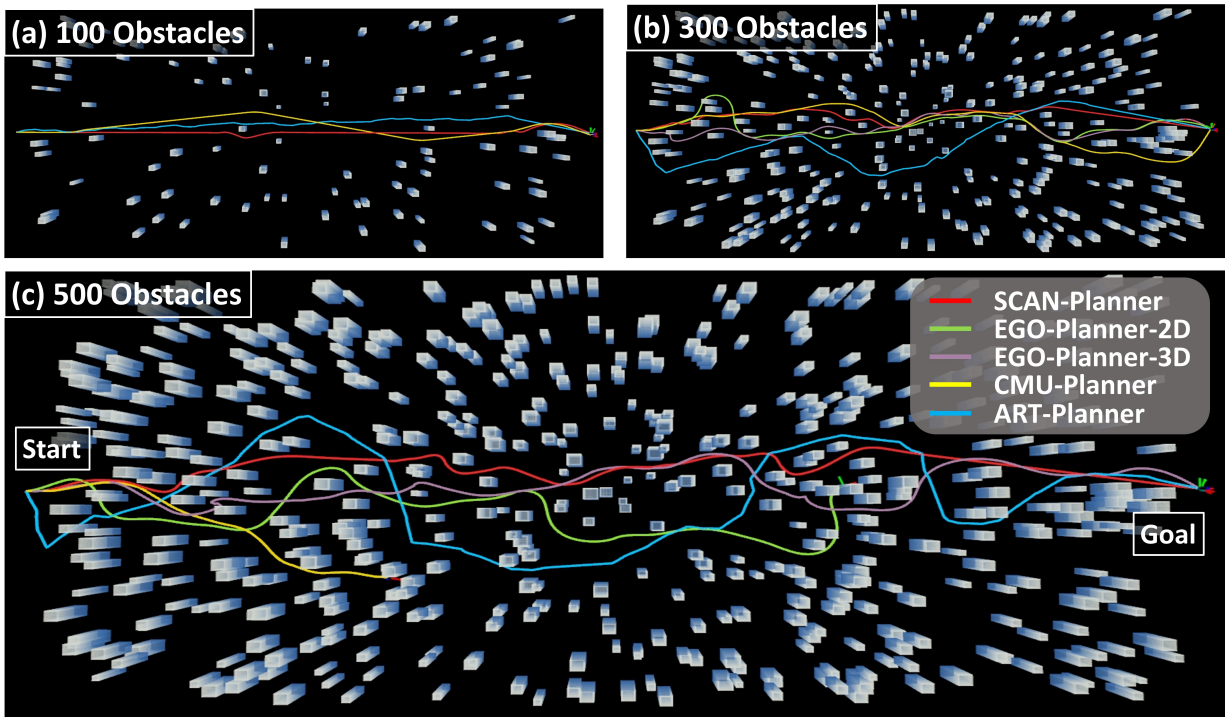

Fig. 7: Simulation comparison in dense random obstacle fields. The planners are evaluated in environments with increasing obstacle density: (a) 100 obstacles, (b) 300 obstacles, and (c) 500 obstacles. The trajectories of SCAN-Planner, EGO-Planner-2D, EGO-Planner-3D, CMU-Planner, and ART-Planner are shown from start to goal. As the clutter density increases, SCAN-Planner generates a smooth and direct collision-free trajectory while maintaining yaw-aware body clearance, whereas the baseline methods exhibit larger detours, unstable route selection, or reduced efficiency in narrow passages.

TABLE II: Quantitative Comparison in Dense Obstacles

Method	SR↑	CR↓	SPL↑
EGO-Planner-2D [4]	0.96	0.04	0.88
EGO-Planner-3D [4]	0.76	0.24	0.75
CMU-Planner [11]	0.92	0.08	0.86
ART-Planner [1]	0.44	0.56	0.31
SCAN-Planner (ours)	**1.00**	**0.00**	**0.95**

B. Dense Obstacles

We first evaluate all planners in random dense-obstacle environments to assess their robustness against local minima and their ability to maintain smooth forward progress. Each map covers 40 m × 20 m, with the number of randomly placed obstacles increasing from 100 to 500. The obstacle width is sampled from 0.2 m to 0.5 m, forming narrow passages with varying clearance margins for the quadruped body.

As shown in Fig. 7, all planners can find feasible paths in sparse scenes, while their efficiency and robustness diverge as the obstacle density increases. EGO-Planner-3D performs collision avoidance in the full 3D free space, which often introduces unnecessary vertical deformation for a ground robot. In contrast, EGO-Planner-2D plans on a planar representation and therefore produces more stable and shorter trajectories in this scenario. ART-Planner relies on random sampling, resulting in piecewise trajectories and frequent detours around distant obstacles. CMU-Planner adopts a sampling-based strategy with a predefined trajectory library. Although the selected trajectory is feasible, it is not necessarily optimal, leading to degraded SPL when narrow passages require precise route selection.

In terms of success rate and collision risk, EGO-Planner-2D, EGO-Planner-3D, and CMU-Planner approximate the robot body using a single inflation radius, which introduces an isotropic safety margin that does not capture the yaw-dependent anisotropy of the quadruped footprint. As a result, conservative inflation can unnecessarily eliminate traversable narrow passages, whereas insufficient inflation may leave residual collision risk in dense clutter. ART-Planner uses a rectangular robot model and thus provides more reliable collision checking, but its lack of trajectory optimization leads to increasingly longer paths as the environment becomes denser. In contrast, SCAN-Planner combines a yaw-aware twin-cylinder footprint with rebound-guided trajectory optimization. This design enables the planner to exploit yaw-dependent safe passages while avoiding excessive detours, maintaining smooth and efficient trajectories even when the number of obstacles increases to 500.

C. 3D Unstructured Scenes

Fig. 8 shows a cluttered desk scene containing overhanging structures and constrained passages. This scene is challenging for planners relying on 2.5D elevation maps, where tabletops are collapsed into a terrain-like representation and the under-table clearance cannot be explicitly constructed. As a result, CMU-Planner treats the desk region as occupied and selects a conservative bypass route. ART-Planner also relies on 2.5D traversability reasoning and random sampling. Since a feasible passage through the constrained interior is not sampled early, the resulting solution detours around the outer wall, yielding the longest path. In contrast, SCAN-Planner maintains a full

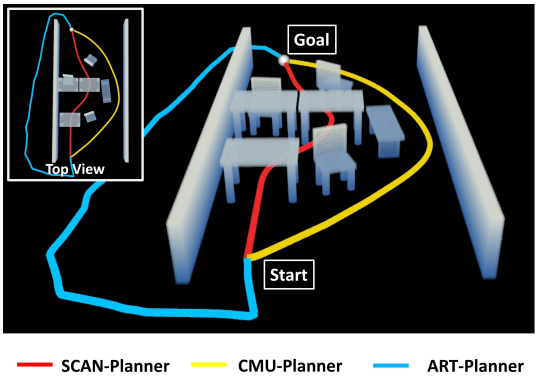

Fig. 8: Comparison in a cluttered desk scene. SCAN-Planner (red) plans through the constrained 3D interior space by considering overhanging obstacles and local body clearance, while CMU-Planner (yellow) and ART-Planner (blue) produce larger detours around the structure.

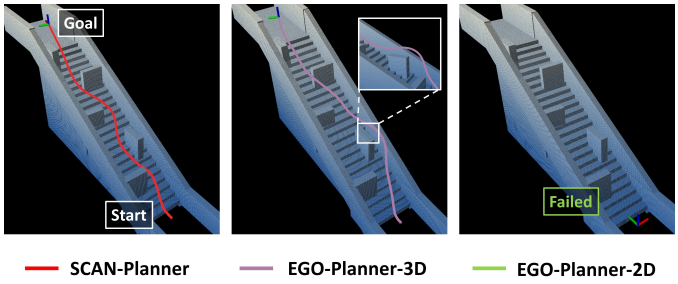

Fig. 9: Comparison in a stair scene with obstacles. SCAN-Planner (red) maintains a stable stair-following trajectory while avoiding obstacles. EGO-Planner-3D (purple) avoids the obstacle with excessive vertical deformation, and EGO-Planner-2D (green) fails because the staircase is projected as an obstacle in its planar representation.

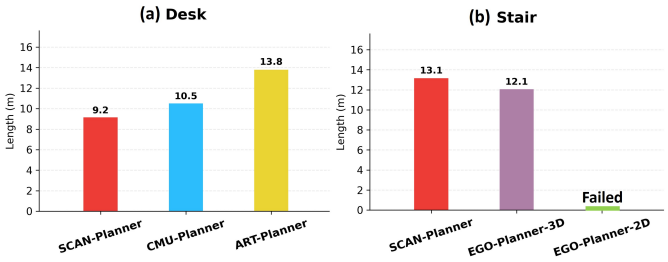

Fig. 10: Path length comparison in representative 3D unstructured scenarios. (a) In the desk scene, SCAN-Planner achieves the shortest feasible path by exploiting the under-table clearance, whereas CMU-Planner and ART-Planner take longer bypass routes. (b) In the stair scene, EGO-Planner-3D yields a shorter path by passing over the obstacle, which is infeasible for contact-supported locomotion. EGO-Planner-2D fails because the staircase is treated as an obstacle in its 2D occupancy representation.

3D occupancy representation and evaluates a yaw-aware 3D footprint of the quadruped body. It can explicitly check the body clearance under the overhanging desk and therefore selects the shortest route through the feasible interior passage.

The stair scenario in Fig. 9 further evaluates vertical traversal and 3D obstacle avoidance. In this case, the planner must avoid the obstacle during stair traversal while preserving the nominal stair-following height profile. EGO-Planner-3D can reach the goal by planning in the full 3D free space. Although it produces a shorter path length in Fig. 10, this reduction is achieved by deforming the trajectory upward and passing over the obstacle, which is infeasible for a ground robot constrained to contact-supported locomotion. When EGO-Planner-2D is used, it produces stable trajectories in planar clutter but cannot model stair traversability, causing the staircase to be treated as an obstacle. In contrast, SCAN-Planner preserves the vertical trend induced by the stairs and suppresses unnecessary rebound in the z-direction. The optimized trajectory therefore avoids the obstacle mainly through horizontal deformation while remaining compatible with ground locomotion.

D. Real-World Experiments

In the real-world experiments, we deploy SCAN-Planner on a Unitree Go2 quadruped robot, as shown in Fig. 1. The robot's dynamic limits are set to $v_{\mathrm{m}} = 1.0$ m/s and $a_{\mathrm{m}} = 0.25$ m/s^2. To obtain dense and high-quality point clouds, the built-in LiDAR of the Go2 is replaced with a Livox Mid-360 LiDAR. An adapted FAST-LIO2 [25] is used for high-frequency state estimation. All perception, mapping, planning, and control modules run onboard an NVIDIA Jetson Orin NX in real time.

We first conduct four indoor experiments in challenging environments to evaluate the local navigation capability of the proposed planner. In the cluttered-room experiment shown in Fig. 11(a), the robot navigates from an entrance to an exit through multiple narrow passages formed by desks and chairs. The results demonstrate that SCAN-Planner can generate safe and efficient collision-free motions in densely cluttered indoor scenes. In the S-shaped bend experiment shown in Fig. 11(b), consecutive sharp turns and confined corridors are used to evaluate trajectory smoothness and robustness. The planned trajectories remain smooth and continuous throughout the task, allowing the robot to traverse the maze-like passage without oscillation or deadlock. We further evaluate the planner in a 3D unstructured environment, as shown in Fig. 11(c), where the scene contains traversable tables, regions blocked by chairs, and low obstacles that can be stepped over. By combining the 3D occupancy representation with yaw-aware body collision checking, the robot can distinguish feasible clearances from blocked regions and select shorter, more efficient paths. Finally, we test the system in dynamic environments with human disturbances, as shown in Fig. 11(d). The log-odds occupancy update enables the local map to adapt to environmental changes, allowing the robot to safely and smoothly avoid slowly moving obstacles during navigation.

To further validate the scalability of SCAN-Planner in long-range navigation tasks, we conduct two large-scale real-world experiments. The first experiment is a cross-floor inspection task in a three-story office building, where predefined inspection waypoints are placed at key locations on each floor. The task covers a volume of approximately $[45 \times 15 \times 15]$ m^3, takes 251 s to complete, and results in a traveled distance of 149 m. A representative point-cloud map and the corresponding 3D navigation trajectory are shown in Fig. 1. During the experiment, the robot reliably traverses staircases and avoids temporary obstacles while maintaining stable navigation performance. The second experiment is a last-mile delivery task in a campus-scale outdoor environment. The navigation region spans approximately $[278 \times 48 \times 5]$ m^3, and the robot travels 367 m in 589 s. A coarse global route is provided by a commercial

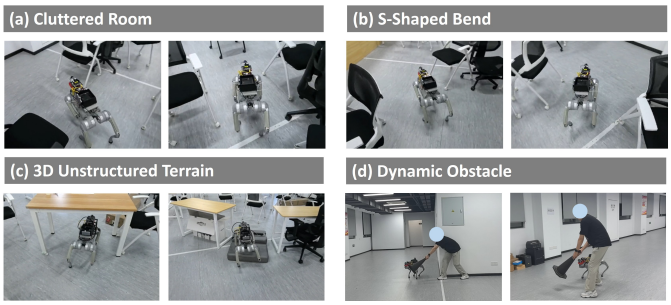

Fig. 11: Indoor real-world experiments of SCAN-Planner in challenging environments. (a) Cluttered room navigation, where the quadruped robot safely passes through narrow passages formed by desks and chairs. (b) S-shaped bend navigation, where consecutive sharp turns and confined corridors evaluate the smoothness and robustness of the planned trajectory. (c) 3D unstructured terrain traversal, where the robot reasons about tables, chairs, and low obstacles to distinguish traversable and non-traversable regions. (d) Dynamic obstacle avoidance, where the planner updates the local occupancy map online and generates safe motions around pedestrians and other moving obstacles.

navigation map, while SCAN-Planner performs local planning. During the task, the robot safely avoids pedestrians, bicycles, and large vehicles, and reaches the destination efficiently. These results show that the proposed sliding-map representation enables accurate local obstacle avoidance while remaining scalable to long-range navigation in large-scale indoor and outdoor environments. More detailed experimental results and visualizations are provided in the **supplementary video**.

VII. Conclusion

This paper presented SCAN-Planner, a spatial collision-aware local planning framework for route-guided long-range quadruped navigation. The proposed method addresses three practical requirements for deploying quadruped robots in cluttered and unstructured environments: body-aware motion in narrow spaces, stable 3D planning for ground robots, and scalable local planning under long-range route guidance. To this end, SCAN-Planner introduces a yaw-aware twin-cylinder footprint for efficient whole-body collision checking, a projected A* rebound search with z-gradient suppression for horizontally dominated obstacle avoidance, and a robot-centric sliding map with dead-end recovery for high-resolution large-scale navigation. Simulation and real-world experiments demonstrate that the planner generates smooth, safe, and efficient trajectories in dense obstacle fields, overhanging 3D scenes, stair environments, and long-range indoor and outdoor tasks. Future work will further incorporate terrain analysis and autonomous exploration capabilities, enabling goal-free 3D scene navigation without predefined target points.

References

[1] L. Wellhausen and M. Hutter, "ArtPlanner: Robust legged robot navigation in the field," *Field Robotics*, vol. 3, pp. 413–434, 2023.

[2] C. Chen, J. Frey, P. Arm, and M. Hutter, "SMUG planner: A safe multi-goal planner for mobile robots in challenging environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7170–7177, 2023.

[3] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, "Perceptive locomotion through nonlinear model-predictive control," *IEEE Transactions on Robotics*, vol. 39, no. 5, pp. 3402–3421, 2023.

[4] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "Ego-planner: An esdf-free gradient-based local planner for quadrotors," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 478–485, 2021.

[5] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3529–3536, 2019.

[6] J. Tordesillas, B. T. Lopez, M. Everett, and J. P. How, "FASTER: Fast and safe trajectory planner for navigation in unknown environments," *IEEE Trans. Robot.*, vol. 38, no. 2, pp. 922–938, 2022.

[7] S. Geng, Q. Wang, L. Xie, C. Xu, Y. Cao, and F. Gao, "Robo-centric esdf: A fast and accurate whole-body collision evaluation tool for any-shape robotic planning," in *IROS*. IEEE, 2023, pp. 290–297.

[8] Y. Son, H. Lee, H. Kang, J. Park, S. Nam, J. Oh, B. Yi, H. Yu, and H. R. Choi, "3D RoA-Planner: Path planner for quadruped robots in confined spaces using 3D rotatable areas," *IEEE Robotics and Automation Letters*, vol. 11, no. 6, pp. 7436–7443, 2026.

[9] M. Zhang, Z. Tian, Y. Xia, C. Xu, F. Gao, and Y. Cao, "Efficient trajectory generation based on traversable planes in 3D complex architectural spaces," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 14 513–14 519.

[10] Y. Li, K. Chen, Y. Wang, W. Zhang, J. Wang, H. Chen, and Y. Liu, "Real-time multilevel terrain-aware path planning for ground mobile robots in large-scale rough terrains," *IEEE Transactions on Robotics*, vol. 41, pp. 4159–4179, 2025.

[11] C. Cao, H. Zhu, F. Yang, Y. Xia, H. Choset, J. Oh, and J. Zhang, "Autonomous exploration development environment and the planning algorithms," in *2022 IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 8921–8928.

[12] Y. Zhang, X. Chen, C. M. Feng, B. Zhou, and S. Shen, "FALCON: Fast autonomous aerial exploration using coverage path guidance," *IEEE Transactions on Robotics*, vol. 41, pp. 1365–1385, 2025.

[13] Y. Ren, Y. Cai, F. Zhu, S. Liang, and F. Zhang, "ROG-Map: An efficient robocentric occupancy grid map for large-scene and high-resolution lidar-based motion planning," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 8119–8125.

[14] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3259–3278, 2022.

[15] Z. Han, Z. Wang, N. Pan, Y. Lin, C. Xu, and F. Gao, "Fast-racing: An open-source strong baseline for SE(3) planning in autonomous drone racing," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 8631–8638, 2021.

[16] B. Zhou, J. Pan, F. Gao, and S. Shen, "RAPTOR: Robust and perception-aware trajectory replanning for quadrotor fast flight," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1992–2009, 2021.

[17] L. Wellhausen and M. Hutter, "Rough terrain navigation for legged robots using reachability planning and template learning," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 6914–6921.

[18] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, "Elevation mapping for locomotion and navigation using GPU," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 2273–2280.

[19] J. Frey, D. Hoeller, S. Khattak, and M. Hutter, "Locomotion policy guided traversability learning using volumetric representations of complex environments," in *IROS*. IEEE, 2022, pp. 5722–5729.

[20] Y. Dong, J. Ma, L. Zhao, W. Li, and P. Lu, "MARG: MAstering risky gap terrains for legged robots with elevation mapping," *IEEE Transactions on Robotics*, vol. 41, pp. 6123–6139, 2025.

[21] C. Li, S. Lin, S. Qu, Z. Liu, Q. Yang, M. Q.-H. Meng, and Y. Sun, "Stable trajectory planning for quadruped robots using terrain features at feet end," *IEEE Robotics and Automation Letters*, vol. 11, no. 2, pp. 2266–2273, 2026.

[22] B. Tang, Y. Ren, Y. Cai, F. Kong, W. Liu, F. Zhu, L. Yin, L. Shi, and F. Zhang, "Memory-efficient boundary map for large-scale occupancy grid mapping," *The International Journal of Robotics Research*, p. 02783649261425266, 2026.

[23] F. Kong, X. Liu, B. Tang, J. Lin, Y. Ren, Y. Cai, F. Zhu, N. Chen, and F. Zhang, "MARSIM: A light-weight point-realistic simulator for lidar-based uavs," *arXiv preprint arXiv:2211.10716*, 2022.

[24] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Mathematical Programming*, vol. 45, no. 1, pp. 503–528, 1989.

[25] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lio2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.